IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 8 December 2024, accepted 28 January 2025, date of publication 3 February 2025, date of current version 10 February 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3537649

RESEARCH ARTICLE

SensorDBSCAN: Semi-Supervised Active Learning Powered Method for Anomaly Detection and Diagnosis

PETR IVANOV[1], MARIA SHTARK[1], ALEXANDER KOZHEVNIKOV[1], MAKSIM GOLYADKIN[2,3], DMITRY BOTOV[1,4], AND ILYA MAKAROV[1,2,5,6], (Member, IEEE)

[1] AI Talent Hub, ITMO University, 191002 Saint Petersburg, Russia
[2] Artificial Intelligence Research Institute (AIRI), 105064 Moscow, Russia
[3] International Laboratory for Intelligent Systems and Structural Analysis, HSE University, 101000 Moscow, Russia
[4] Department of Computer Science, Neapolis University Pafos, 8042 Paphos, Cyprus
[5] Artificial Intelligence Research Center, National Research Nuclear University MEPhI, 115409 Moscow, Russia
[6] ISP RAS Research Center for Trusted Artificial Intelligence, 101000 Moscow, Russia

Corresponding author: Petr Ivanov (petr_ivanov@niuitmo.ru)

The work of Ilya Makarov was supported by the grant for research centers in the field of Artificial Intelligence (AI) provided by the Analytical Center (ACRF) in accordance with the agreement on the provision of subsidies (identifier of the agreement 000000D730324P540002) and the agreement with National Research Nuclear University Moscow Engineering Physics Institute (MEPhI) No. 70-2023-001309.

ABSTRACT Fault detection and diagnosis (FDD) is a critical challenge in industrial processes aimed at minimizing risks such as safety hazards, costly downtime, and suboptimal production. Traditional supervised FDD methods offer great performance while heavily relying on large volumes of labeled data, whereas unsupervised methods do not depend on labeled data, though are inferior in performance compared to supervised ones. In this paper, we propose SensorDBSCAN, a novel semi-supervised method for anomaly detection and diagnosis. The key innovation lies in achieving good performance with minimal labeled data - less than 1% of the dataset - by leveraging active and contrastive learning techniques. The proposed approach combines a transformer-based encoder trained with a triplet-based contrastive learning objective and the classical density-based clustering algorithm DBSCAN, enabling strong feature extraction, efficient and interpretable feature space organization and simple clustering algorithm. Unlike existing methods, SensorDBSCAN eliminates the need for manual labeling large amounts of data, cluster analysis, and pre-defining cluster numbers, providing greater usability in real-world cases. We validate the effectiveness of our method on the Tennessee Eastman Process (TEP) and its advanced simulations (TEP Rieth and TEP Rieker). SensorDBSCAN demonstrates better performance on well-known and realistic datasets, reducing labeling requirements while maintaining high accuracy of fault detection and diagnostics. The code is available at https://github.com/K0mp0t/sensordbscan.

INDEX TERMS Active learning, semi-supervised learning, time series anomaly detection and diagnosis.

I. INTRODUCTION

Monitoring the condition of industrial equipment is essential to maintaining efficient and reliable production processes. Modern factories increasingly rely on sensor networks to monitor system health and prevent equipment failures. Predictive maintenance strategies have gained prominence for their ability to detect faults early, allowing proactive intervention to minimize downtime, energy consumption, and costs. A 2020 report by the Smart Energy Analytics Initiative highlighted that effective fault detection and diagnosis (FDD) can reduce energy use by 9%, resulting in significant economic benefits [1].

Detecting anomalies alone is insufficient; it is equally important to classify their type to ensure timely and precise responses to potential faults. Identifying the specific type of anomaly not only facilitates root cause analysis but also enables the optimization of fault mitigation strategies. However, many existing anomaly detection methods are limited to signaling the presence of an anomaly without providing insight

The associate editor coordinating the review of this manuscript and approving it for publication was Yu Gu.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

into its nature. This shortcoming significantly reduces their practical applicability, as understanding the characteristics of the anomaly is critical to determining effective corrective actions and preventing recurrence.

Despite their potential, FDD methods face several challenges. Industrial data sets are highly imbalanced, anomalies are rare compared to normal operating states. Moreover, it is impractical to label large amounts of data manually, as this requires significant time and effort. Although supervised learning methods deliver high accuracy, they are constrained by their dependency on extensive labeled datasets. Unsupervised techniques, though less demanding, usually achieve lower accuracy and often require manual cluster analysis or predefined cluster numbers, limiting their scalability and automation. One of such works is SensorSCAN [2], on which our method is based. It uses a transformer encoder [3] for time series data and the SCAN [4] deep clustering technique. Furthermore, existing models struggle to detect and classify new types of anomalies that were not part of the training dataset, a critical need for real-world applications.

Both supervised and unsupervised FDD methods have their weaknesses that limit their usage in real world cases, which makes research of semi-supervised methods for fault detection and diagnosis important: by balancing between great performance of supervised methods and lack of necessity for labeled data of unsupervised ones, we were able to achieve better applicability. We present SensorDBSCAN, a semi-supervised approach for anomaly detection and diagnosis that combines active learning, triplet mining, and classical clustering techniques. We have combined performance similar to that of supervised methods with low needs for labeled data similar to those of unsupervised ones. The novelty of our work lies in its ability to:

1) Achieve high performance with minimal labeled data by leveraging active learning to prioritize the most informative samples for labeling.
2) Use triplet mining and contrastive learning techniques to structure the descriptor space for improved separability, allowing the use of density-based clustering algorithms that do not require labeled data and a predefined number of clusters, for example DBSCAN [5].
3) Eliminate the need for manual cluster analysis, making the system fully automated and adaptable to new anomalies because of the simple descriptor space organization idea taken from facial recognition systems - "descriptors for similar samples ought to be positioned nearby, whereas those for different samples should be placed far apart".

The remaining part of this work is structured as follows: In Section II, we provide an overview of traditional and modern data-driven methods for fault detection and diagnosis. We describe our proposed model in detail in Section III. Dataset information and metrics are explained in Section IV, where we also compare our model with other existing models.

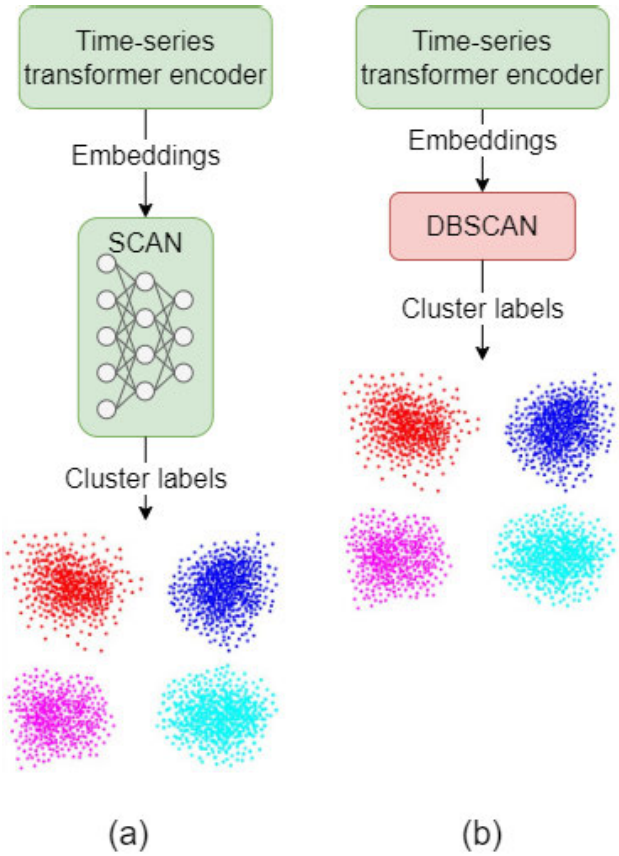


FIGURE 1. Comparison with SensorSCAN [2]. Our method is based on [2] and the key difference is second stage of training: (a) [2] used deep clustering technique with supervised fine-tuning, while our method (b) our method uses semi-supervised triplet training paired with DBSCAN clustering algorithm. Triplet training allows us to make descriptor space separable without fully-supervised training and makes it's manual analysis unnecessary enabling number of clusters automatic estimation.

II. RELATED WORK

There are many methods for the detection and classification of anomalies. These methods can be divided into two categories: supervised and unsupervised. Supervised methods rely on labeled data, which contain the correct answer or classification, while unsupervised methods do not require labeling and are able to find patterns by themselves. The problem with using supervised methods for fault detection is that they are limited to a predefined set of faults that experts have observed. This suggests exploring unsupervised techniques that do not have such limitations. Most modern techniques are data-driven and based on analyzing raw data and interpreting it. Some methods also use bond graph methodologies to provide a physical understanding of systems through the creation of computational models like [6], [7], and [8].

Currently, state-of-the-art methods often use unsupervised or semi-supervised models such as generative adversarial networks (GAN) and long-short-term memory (LSTM) models [9], as well as autoencoders with modified attention layers. These models are chosen because of their ability to take into account long-term dependencies, extract complex features from multidimensional time-series data, and their scalability. This architecture has found widespread application

in anomaly detection tasks, as demonstrated in [10]. However, recent studies [11] have shown the superiority of transformers over CNN and RNN-based methods. For our method, we chose to use a transformer-based encoder because of its ability to efficiently process time series data and its suitability for large-scale applications.

Modifications to LSTM are common [12], [13], [14]. They have strengthened feature attention and self-attention blocks. Probabilistic neural networks [15] also show good performance. In [16], many papers have shown that "out-of-the-box" solutions are not suitable for anomaly detection. "Out-of-box deep learning strategies may yield impressive accuracy results, but they are poorly suited for real-world applications." This is why it is common practice to add feature attention blocks and develop new self-attention blocks [12], [13], [17]. Models also benefit from using dilated convolution modules [18]. Few-shot learning is a machine learning technique that allows an algorithm to make precise predictions based on a very small number of labeled examples. This technique is useful when you have limited or insufficient data. In [19], the authors used few-shot learning in combination with a transformer encoder and variational attention.

Convolutional neural networks are widely used as feature extractors for fault detection methods: initially used to extract features from images, they were later adapted for use in fault detection [20], [21], where one-dimensional CNNS (2)-DCNN) have been shown to have the ability to deal with challenging tasks. Their ability to learn local patterns makes them highly useful with time-series data, which is a common data domain for fault detection tasks. In recent years, there have been many different versions and modifications of CNNs for fault detection [22], [23], [24], which have shown great performance in supervised settings, and with the growing interest in unsupervised models in various domains [25], [26], [27], [28], [29], unsupervised, self-supervised, and semi-supervised CNN-based approaches were also studied [30], [31], [32], [33]. The advantages are the following: high classification accuracy and computationally inexpensiveness. However, when data lack local patterns, have noise, or time-series samples become long, CNN-based methods may perform poorly. It also has been noted that despite the fact that there are many works on modifications of CNNS for fault detection, they are rarely used in production.

Another interesting method [34] uses the association discrepancy for anomaly detection. It compares the prior association (local-range prediction) and the series association (long-range prediction) to determine if an anomaly is occurring. The idea is that normally these two predictions are different, but in the case of an anomaly, they become similar. The authors also show how to improve the distinguishability between normal and abnormal points to obtain better results.

Online learning is also used in some fault detection methods. It is a method of machine learning in which data is made available sequentially, one observation at a time, as opposed to offline learning, where we train a model on the entire dataset. This method is data-efficient and adaptable, which is useful for time series prediction [35], [36], [37].

In the context of FDD in manufacturing, human-in-the-loop (HITL) machine learning has emerged as a promising approach to improve model performance and adaptability [38]. These methods facilitate a collaborative relationship between human experts and machine learning algorithms, leading to more effective anomaly detection and diagnosis. By incorporating human feedback into the learning process, HITL techniques can enhance the model's ability to identify anomalies, especially in scenarios with limited labeled data. This collaboration between human expertise and machine learning is essential for creating adaptive solutions that can respond to new and unexpected anomalies in manufacturing processes.

The use of active learning approaches for anomaly detection has received a lot of attention in recent studies. An interesting technique is to use semi-supervised autoencoders with online active learning [35]. Given the high cost of the labeling process, active learning proves to be beneficial in such cases, as it can select the most informative data for training. The authors combine a streaming-based on-line learning scenario with active learning and use the Tennessee Eastman Process benchmark for comparison. Active learning has also proven to be a robust method for detecting anomalies in the Internet of Things, helping to achieve notable levels of performance [39] without the use of traditional static and rule-based methods. Active learning is also commonly used in cases where users regularly encounter new, unknown types of anomalies [40]. You can also use active learning to enhance your existing pipeline and get better results. As shown in [41], active learning helps to increase the performance of a model on both labeled and unlabeled data by up to 90 percent compared to the baseline. In another study [42] authors demonstrate that a hybrid model combined with expert labeling yields increased performance.

Overall, active learning has proven to work well in our case, where we have to deal with a semi-supervised setting and the possibility of getting faults that were not present in the training dataset.

III. PROPOSED METHOD

Most modern AI-powered anomaly detection systems are either supervised or unsupervised: the first ones require large amounts of labeled data, which makes their integration expensive and time-consuming, while the second ones may perform significantly worse. We have tried to find the middle point between those two - the semi-supervised method, combining decent performance with low integration costs. We were able to make our method use less than a thousand labeled samples while showing a performance that is good enough to solve real-world problems. Our method has two major training stages: self-supervised pretraining which is depicted in Figure 2 (a) and semi-supervised training shown in Figure 2 (b).

Active learning techniques allow us to efficiently select a small subset of data for labeling paired with triplet-based

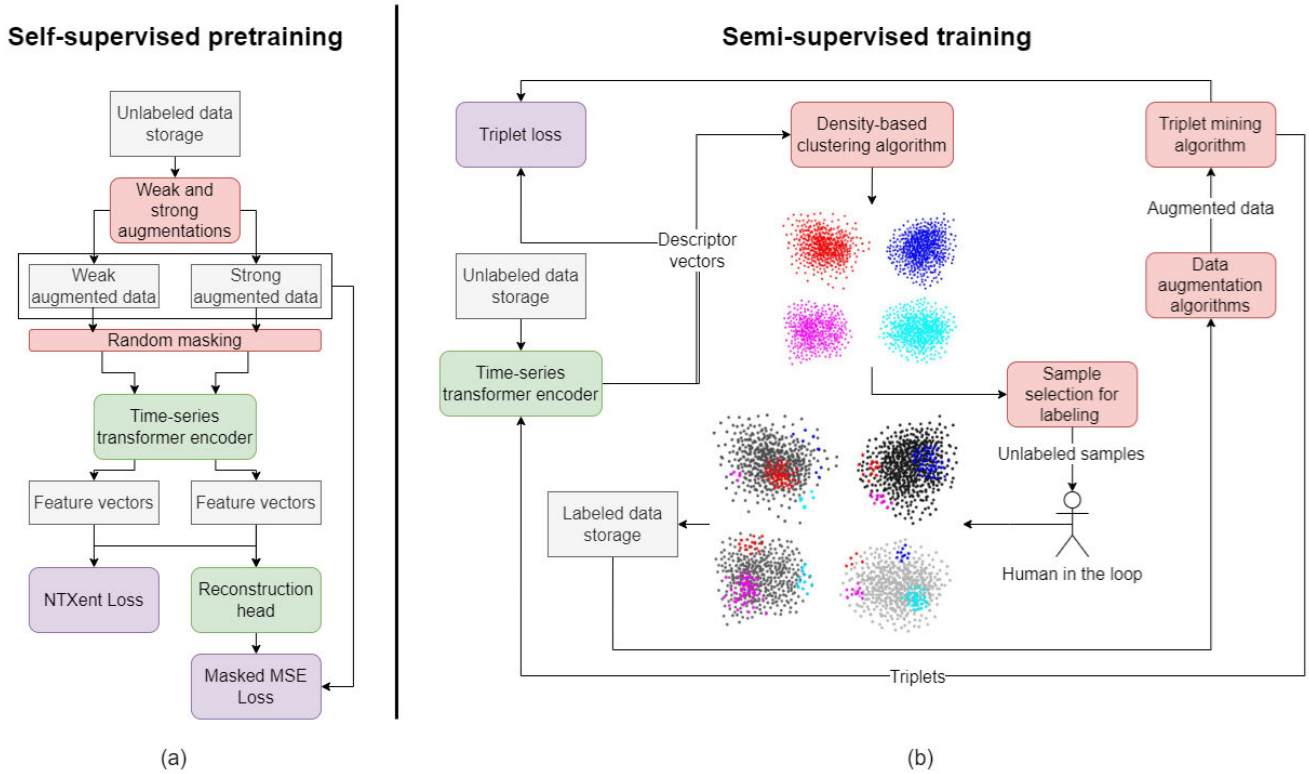


FIGURE 2. Method overview. The first stage (a) is similar to [2]: two sets of augmentations with two training objectives—masked sequence reconstruction and self-supervised contrastive learning. The second stage (b) is semi-supervised: DBSCAN produces cluster labels for descriptor vectors from transformer encoders based on known data, and cluster geometry samples for labeling are selected. After the human labeling and automated augmentations, labeled data is used to construct, rank, and select the best triplets to train encoders with triplet loss.

active learning, which provides a significantly larger number of efficient triplets as training data.

As we use the triplet loss function, our transformer-based encoder has gained properties similar to those of siamese neural networks [43]. Siamese networks trained with triplets produce strong descriptors (representation vectors) for previously unseen data, which allows us to use a subset of real fault types for training and makes our method robust to new data and fault types. These properties have been widely explored in the field of image matching and human face recognition [44], [45].

A. PRE-TRAINED ENCODER

We employ self-supervised learning 2 (a) to pretrain our transformer-based [3] encoder, which allows it to learn descriptors (embeddings for infinite data) that separate different samples from each other in the descriptor space. Well-organized pretraining is the key; it makes the model learn input data structure and internal patterns without labeled data.

A transformer-based model is highly useful for sequence data processing because of its self-attention module, which is able to find better internal relations than previously used methods (e.g., convolutional autoencoders, recurrent neural networks, etc.). Our transformer-based feature extractor consists of four main parts: encoder $\mathcal{T}$, sequential pooling layer $\mathcal{P}$, projection head $\mathcal{H}$, and reconstruction head $\mathcal{R}$. The encoder $\mathcal{T}$ is a three-layer Transformer encoder with the embedding dimensionality H. It maps a sensor data sample of length L with D features to a sequence of embedding vectors of the same length, $\mathcal{T}\colon \mathbb{R}^{L\times D} \to \mathbb{R}^{L\times H}$.

We use the RoPE positional encoding mechanism [46], which has shown great performance in a number of tasks. To map the sequence of embedding vectors $\tilde{h} = [h_1, \ldots, h_L] \in \mathbb{R}^{L\times H}$ into a single vector $\hat{h} \in \mathbb{R}^H$ we use the sequence pooling mechanism $\mathcal{P}$: it uses the attention-like principle to assign weight to each embedding vector in sequence and calculate the weighted sum:

$$w = \mathrm{softmax}(W_{pool}^{\top}\tilde{h}^{\top}),\ W_{pool} \in \mathbb{R}^{H\times 1}, \tag{1}$$

$$\hat{h} = w\tilde{h}. \tag{2}$$

The projection head $\mathcal{H}$ is a simple 2-layer MLP with intermediate BatchNormalization [47] and ReLU activation. It allows us to reduce the final embedding dimensionality and encode input information in embedding vectors efficiently $\mathcal{H}\colon \mathbb{R}^H \to \mathbb{R}^F$, where F is the size of feature extractor embeddings.

Finally, the reconstruction head $\mathcal{R}$ is used to reconstruct the raw input data from the embedding vectors of the encoder $\mathcal{R}\colon \mathbb{R}^H \to \mathbb{R}^D$. It is used exclusively for pretraining as a part of an additional pretraining task to better pre-organise embedding space and consists of one linear layer.

The first self-supervised pretraining task is the reconstruction of masked data. Such reconstruction tasks are

widely employed in autoencoder neural networks and have shown great applicability to learning internal data structure. Moreover, masking data for reconstruction makes the model robust to sensor malfunctions, which may occur in real-world usage cases. We use the mean squared error as a loss function for this task, whose values are calculated only for masked sequence elements.

$$\mathcal{L}_{rec}(\hat{X}, X) = \frac{1}{B}\sum_{i=1}^{B}\frac{1}{|M^i|}\sum_{l,d\,:\,M^i_{ld}=0}(\mathcal{R}(\mathcal{T}(\hat{X}))_{ld} - X_{ld})^2, \tag{3}$$

where B is the batch size, M^i is number masked values for sample i and l and d correspond to the number of timestamps and the number of sensors, respectively.

For the second self-supervised pretraining task, we construct two different sets of samples with different sets of augmentations. We use contrastive learning techniques whose goal is to make embeddings of similar samples closer to each other in the embedding space than to any other different sample. Using this task for pretraining, we make our model learn to find common patterns in data samples and separate samples into different groups where each group consists of similar samples.

We used two augmentation algorithms to constrain pairs of samples: a weak augmentation algorithm and a strong one. The weak augmentation algorithm consists of jitter and scaling augmentations, which may also be described as additive and multiplicative augmentations: for jitter (additive) augmentation, we add Gaussian noise to input data, and for scaling (multiplicative) augmentation, we sample values from normal distribution with mean μ_{weak} close to 1 and variance σ^2_{weak} close to 0, then we multiply each element of input data with sampled values. The strong augmentation algorithm consists of scaling and permutation augmentations: the scaling mean for the strong augmentation is lower than that for the weak one, $\mu_{strong} \approx 0.5$, and the scaling variance is the same. The permutation is performed by splitting the original time series sequence into a random number of subsequences sampled from *U(1, 17)* and randomly shuffling them together.

For this pretraining task, we use the NT-Xent loss function [48], which helps us efficiently prepare the descriptor space without the need for labeled data.

$$l_{i,j} = -\log\frac{\exp(\mathrm{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2B}[k \neq i]\exp(\mathrm{sim}(z_i, z_k)/\tau)}, \tag{4}$$

where $z_i, z_j, z_k \in \mathbb{R}^F$ denotes the projection head outputs, $\mathrm{sim}(\mathbf{u}, \mathbf{v}) = \mathbf{u}^\top\mathbf{v}/(||\mathbf{u}||\ ||\mathbf{v}||)$ denotes the cosine similarity, and τ denotes the temperature parameter. The final loss $\mathcal{L}_{cont}$ is calculated across all the $2B$ positive pairs.

B. SAMPLES SELECTION FOR LABELING

Active learning is used epoch-wise to select a data subset of a certain size N to be labeled. We used the Calinsky-Harabasz score to estimate whether we should sample data for labeling at each epoch. For further details, please refer to subsection III-E. We employ the active learning technique here to minimize the number of samples to be labeled. In practical usage with unseen data, we would have to use expert data labeling, so by lowering amounts of data to be labeled, we save time, effort, and money. For labeling, we used time series segments of fixed length W, each assigned a classification label. During experiments, we have simulated expert labeling with labels.

We sample data for labeling in a few steps. In the initial epoch, where no labeled data is available, we calculate the sampling weight $w_i^{(0)}$ for the cluster c_i based on the cluster size n_i.

$$w_i^{(0)} = \frac{n_i}{\sum_{j=1}^{k} n_j} \tag{5}$$

For subsequent epochs, where labeled data was available, the sampling weight $w_i^{(t)}$ for the cluster c_i at epoch t is calculated by estimating the number of misclassified samples $m_i^{(t-1)}$ for the cluster c_i from the previous epoch:

$$w_i^{(t)} = \frac{m_i^{(t-1)}}{\sum_{j=1}^{k} m_j^{(t-1)}} \tag{6}$$

We calculate label statistics for each cluster based on labels available at the moment, the most popular label is estimated for each cluster, and cluster samples that do not belong to the most popular label for given cluster are considered misclassified. Using this technique, we can estimate the variance of the label for each cluster and assign higher sampling weights to clusters with a greater variance of the label.

Based on calculated weights, we randomly select a cluster c_i with probability proportional to w_i^t. We sample one element from this cluster by constructing a probability density function based on the cluster's mean μ_i and variance σ_i^2 divided by 4 to select random, though, more typical points of cluster for labeling.

$$p_i(x) = \frac{1}{\sqrt{\pi\sigma_i^2}}\exp\left(-\frac{(x-\mu_i)^2}{\sigma_i^2/4}\right) \tag{7}$$

C. TRIPLET MINING

We build triplets of time series segments using a previously labeled subset of training data. We use several window sizes during triplet building: original samples are being split into smaller ones, which allows us to minimize the number of samples to be labeled at every epoch. For example, if we use windows of sizes 50, 75, and 100 with a step size of 5, we get 18 times more samples than the original data had. Each triplet consists of three elements: anchor sample (A), positive sample (P), and negative sample (N). The number of triplets increases drastically with the number of labeled data during training. We apply another active learning algorithm composed of three steps described in the following to select the N best triplets for training at each epoch.

First, we calculate the triplet distance with a simple yet effective Euclidean distance function. The distance is

calculated between anchor, positive and negative sample descriptors and is described as

$$D(a, p, n) = ||\mathcal{F}(a) - \mathcal{F}(n)||_2^2 - ||\mathcal{F}(a) - \mathcal{F}(p)||_2^2 \quad (8)$$

where $\mathcal{F}(x) = \mathcal{H}(\mathcal{P}(\mathcal{T}(x)))$.

Next, we filter for semihard triplets to ensure smoother convergence: extreme triplet distances may cause high loss and gradient values. This filtering is performed by selecting triplets that satisfy the following condition:

$$D(a, p, n) > 0 \quad (9)$$

Finally, we order triplets by their distance $D(a, p, n)$, where the lowest triplet distance indicates the most difficult triplet. We select N hardest triplets for training; this allows the encoder $\mathcal{T}$ to focus on the hardest cases, resulting in improved overall performance and faster convergence.

D. TRIPLET TRAINING AND DESCRIPTOR SPACE ORGANIZATION

We use a triplet loss with a margin parameter ϵ_t to organize the descriptor space. It aims to organize descriptor space so that descriptors for time series segments with the same labels lie in each other's Epsilon neighborhood and differently labeled time series segments are located distantly. One of the core advantages of using triplet-based contrastive learning is reducing the amount of training data: we can construct significantly more triplets from a small-sized dataset (e.g., from 1000 samples distributed uniformly across 25 classes we get $3.66 * 10^7$ triplets). The same approach is widely used in, e.g., face recognition tasks, where modern methods are able to correctly learn datasets with a large number of unique classes and, moreover, show great performance with new unseen data.

$$\mathcal{L}_t(a, p, n) = \frac{\sum_{i=1}^{N} \max\left(d(a_i, p_i) - d(a_i, n_i) + \epsilon_t, 0\right)}{N} \quad (10)$$

where ϵ_t is margin hyperparameter $d(s_1, s_2) = ||\mathcal{F}(s_1) - \mathcal{F}(s_2)||_2^2$ and $\mathcal{F}(x) = \mathcal{H}(\mathcal{P}(\mathcal{T}(x)))$.

Triplet loss was paired with additional optimization criteria for minimizing the distance between samples belonging to the same class, ensuring faster convergence and, as our experiments have shown, better reduction of the outlier cluster during training. Furthermore, it helps produce more dense clusters, which is also helpful for density-based clustering algorithms. The value of this additional loss function is weighted with a hyperparameter $\alpha = 0.01$ that guarantees that the loss of triplets will be the main criterion for training. The full loss function is described below:

$$\mathcal{L}(a, p, n) = \mathcal{L}_t(a, p, n) + \alpha * \frac{\sum_{i=1}^{N} d(a_i, p_i)}{N} \quad (11)$$

where $d(s_1, s_2) = ||\mathcal{F}(s_1) - \mathcal{F}(s_2)||_2^2$ and $\mathcal{F}(x) = \mathcal{H}(\mathcal{P}(\mathcal{T}(x)))$.

By organizing the descriptor space in the way described above, we solve three problems:

1) We are now able to work with an undefined number of fault types (or system states). The well-organized descriptor space can be successfully clustered with simple density-based clustering algorithms that do not require a predefined number of clusters.
2) We believe that when a new fault is observed, its descriptors will differ from those seen before, so it may form a new cluster containing data belonging to the newly observed state. This approach makes model tuning unnecessary when a new system state is present and significantly improves model generalization and novelty adaptation ability.
3) Using triplet loss with a margin parameter ϵ_t, we can control the density of the descriptor space, which is extremely helpful for classical clustering density-based methods, e.g. DBSCAN, OPTICS, and their variations. We can synchronize the triplet loss margin parameter ϵ_t and the DBSCAN parameter ϵ_c since they share the same purpose of determining the distance between samples from different classes.

After the encoder is well trained, we can get good results with the simple DBSCAN algorithm using an epsilon parameter equal to or slightly less than the triplet loss margin parameter ϵ_t.

E. CLUSTERING AND TRAIN PROCESS EVALUATION

After each epoch, we recalculate the descriptors for the entire training set and perform clustering. Clustering results are then evaluated using the Calinski-Harabasz index, which allows us to track the descriptor space evolution process. The Calinski-Harabasz index measures the ratio between intracluster and intercluster variance. For a given data set of n points: $\{x_1, \ldots, x_n\}$, and the assignment of these points to k clusters: $\{C_1, \ldots, C_k\}$, the Calinski—Harabasz index (CH) is defined as:

$$\mathrm{CH} = \frac{\sum_{i=1}^{k} n_i ||(c_i - c)||^2}{\sum_{i=1}^{k} \sum_{x \in C_i} ||(x - c_i)||^2} \cdot \frac{n - k}{k - 1} \quad (12)$$

where n_i is the number of points in cluster C_i, c_i is the centroid of C_i, and c is the overall centroid of the data. This index is not normalized, though we can use it to track progress through the training process and decide on each epoch whether we should label more data or not. Unlike other clustering evaluation methods, e.g., the silhouette score, the Calinski-Harabasz score does not require any additional parameters; its value is based entirely on geometrical and topological cluster features.

We start sampling and labeling new samples only after the Calinski-Harabasz score growth stops, which allows us to lower the number of labeled samples even more.

F. OUTLIER HANDLING

Density-based clustering methods, for example, DBSCAN [5], OPTICS [49], and their variations, may produce additional outlier clusters. This cluster contains data points that are either too far from existing clusters to be assigned to them or too sparse to form a cluster on their own. Even though these points are assigned to one cluster, they do not form a

"correct" cluster. Our experiments have shown that if we use simple neighbor-based methods to assign these samples to the closest clusters, we will be able to further improve the method's performance.

In our method, we have used the simple k-neighbor method [50] for this assignment. As a result, we reduced the number of clusters by one, although we fixed some encoder errors. If new fault-type samples are present, they will form a cluster on their own and will not be impacted by this outlier handling mechanism, thus not being incorrectly assigned to any existing cluster.

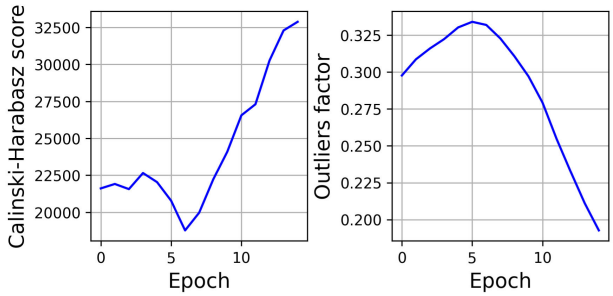


FIGURE 3. Epoch-wise training process parameters for TEP_{Ricker}. Outliers percentage reduces rapidly during the training process while Calinsky-Harabasz score increases which indicates descriptor space topological quality growth leading to better clustering results.

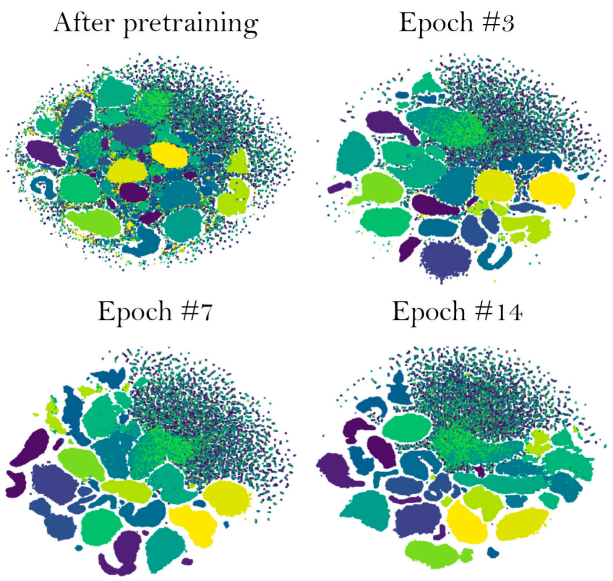


FIGURE 4. Descriptor space evolution process t-SNE visualization for TEP_{Rieth}. Left-to-right top-to-bottom epochs 0, 3, 7, 14. Through the training process descriptor space becomes significantly less noisy, while clusters become better separable.

IV. EXPERIMENT DETAILS

A. DATASETS

The Tennessee Eastman Process (TEP) is a widely recognized benchmark for evaluating process control and FDD methods. The data set presented in [51] is based on an actual chemical plant process, simulating various operations and faults. There are two distinct numerical simulators available for the TEP.

The first employs the control scheme from [52]. The data set with 20 faults was presented in [53]. In order to improve the variety of data, an extended version[1] of this data set (hereafter TEP_{Rieth}) was implemented in [54]. The second simulator, based on different control schemes [54], [55], [56], is available from the Tennessee Eastman Challenge Archive[2] and includes the modified TEP model from [57] (hereafter TEP_{Ricker}). This model addresses issues with the simulation results depending on the chosen ODE solver when random disturbances are active and adds 8 faults, process measurements, and simulation options. The TEP_{Ricker} is detailed in [58].

In TEP_{Rieth}, there are 500 simulation runs for each state of the process in both the training and the testing sets, for a total of 21,000 runs. Each run includes 500 timestamps and 960 for training and testing, respectively, at a sampling rate of 3 minutes. The training set was unbalanced by cropping it to 500 normal runs and 5 runs each for the faulty states, while the testing set remained unchanged. TEP_{Ricker} includes 2,000 timestamps per run, with a 3-minute sampling rate covering 100 hours, with faults introduced at the 30th hour. It consists of 100 runs per process state, totaling 2,800. In order to account for the lack of a train/test split in the original dataset, we divided the simulation runs using an 80/20 ratio. We did not use all the markup for training from the training dataset, but we used all training data for pretraining.

B. METRICS

We use clustering methods to solve the classification problem, so we evaluate both the clustering and classification quality using two sets of metrics.

1) CLUSTERING METRICS

We use several metrics to compare our method with other methods and better understand the clustering performance. As the main quality metric for clustering, we use unsupervised clustering accuracy (ACC), which evaluates the accuracy of the best possible labeling.

$$\mathrm{ACC}(Y, C) = \max_{m} \frac{\sum_{i=1}^{n} [y_i = m(c_i)]}{n}, \tag{13}$$

where m is the clustering-labels matching function, n is the size of the used dataset.

The Adjusted Rand Index (hereafter ARI) [59] is used because it has good interpretability: ARI is proportional to the number of sample pairs whose labels are the same in both predicted cluster labels and target ones or are different in both. The adjusted Rand index is the adjusted for chance version of the regular index.

$$\mathrm{RI} = \frac{a + b}{\binom{n}{2}} \tag{14}$$

where a is the number of samples correctly assigned to the same clusters, b is the number of samples correctly assigned

[1] https://dataverse.harvard.edu/dataset.xhtml?persistentId=doi:10.7910/DVN/6C3JR1

[2] http://depts.washington.edu/control/LARRY/TE/download.html

to different clusters, and n is the number of samples.

$$\mathrm{ARI} = \frac{\mathrm{RI} - E(\mathrm{RI})}{1 - E(\mathrm{RI})}, \tag{15}$$

where $E(\cdot)$ is the expected value.

Among ACC and ARI, we use Normalized Mutual Information (hereafter NMI) [60] to measure the agreement value between the predicted label assignment and the target assignment.

$$\mathrm{NMI}(Y, C) = \frac{2I(Y, C)}{H(Y) + H(C)}, \tag{16}$$

where $I(\cdot, \cdot)$ is the mutual information function and $H(\cdot)$ is the entropy function.

2) CLASSIFICATION METRICS

For classification quality evaluation, we use a simple but representative true positive rate (hereafter Detection TPR) and a false positive rate (hereafter Detection FRP). However, we consider faulty states as positives and normal states as negatives, so TPR indicates the proportion of correctly detected faulty time series samples and FPR indicates the proportion of normal time series samples labeled as faulty.

In binary fault detection, we evaluate the quality of fault diagnosis (fault type classification) with the Correct Diagnosis Rate (hereafter CDR), which indicates the proportion of correctly classified faults.

And finally, due to the practical orientation of our method and the time series nature of the data, we use the average detection delay (hereafter ADD), which indicates the average anomaly detection delay. This metric allows us to demonstrate the usability of the method in practical cases. Knowing the time step of the input data, we are able to calculate ADD using real-world time units, which might be extremely useful in real-world situations.

C. EXPERIMENT DETAILS

We trained a self-supervised encoder with fixed length $\mathcal{W} = 100$ windows in two steps: self-supervised pretraining and semi-supervised triplet-based contrastive learning. Similar model hyperparameters were used for both datasets. We have used 20% of the training subset for self-supervised pretraining and semi-supervised learning and the full test subset for evaluation.

During the labeling phase, 50 samples are selected epoch-wise for labeling. For both datasets, we have done self-supervised pretraining for 7 epochs and semi-supervised contrastive training for 15 epochs. For details of self-supervised pretraining and hyperparameters, refer to Section III-A. Batch size equal to 1280 was used during semi-supervised triplet training; for each epoch we have selected 100 batches of the best triplets for training; refer to Section III-C for details of the triplet selection algorithm. A margin value ϵ_t of 1 was set for the triplet loss function and the ϵ_c hyperparameter for the DBSCAN algorithm was set to 0.9 to better separate clusters due to the triplet loss function value being close but not equal to zero. For outlier processing, the k-nearest neighbors algorithm with $k = 50$ was used. Full experimental results and configurations can be found in the GitHub repository, where the code for our method will be published, which we have designed for the reproducibility of the full results.

In our experiments, we used a NVIDIA GeForce 3090 GPU with 24 GB of VRAM, 64 GB of DDR5 RAM, and a 6-core CPU. It takes about 3 hours to run the training and approximately 10 minutes for evaluation. We have used a light-weight transformer as a feature extractor, though, DBSCAN is a computationally intensive algorithm, making its usage undesirable in cases that require real-time inference speed. Even with GPU-specific optimizations [61], DBSCAN is not capable of real-time inference on such amounts of complex data. However, for a real world usage of pretrained models, incremental inference is possible: cluster labels obtained during initial run may be used for classification of new data points with the K-nearest neighbors algorithm, which allows our method to be used in scenarios that require real-time anomaly detection.

D. BASELINES

We have conducted a few experiments with unsupervised models to display the supremacy of the semi-supervised method. We took a simple combination of PCA and Kmeans [53] as an elementary scheme with low computational complexity. We have also considered more advanced methods: ST-CatGAN [62], ConvAE [63], and SensorSCAN [2].

PCA is used to create representations for sensor data in a 25-dimensional embedding space, which are clustered with Kmeans later. PCA parameters were estimated with a grid search algorithm based on the detection TPR of the whole algorithm.

ST-CatGAN has two main parts: a short-time Fourier transform is used to construct a three-dimensional matrix of size $(16, 16, D)$ from the original $(100, D)$ time series sample, where D is the number of features in the input data. For the second part, a modified version of the generative adversarial network [64] is used to classify samples. The key difference between CatGAN and original GAN lies in the discriminator part: unlike the GAN discriminator that does binary classification, the CatGAN discriminator does multiclass classification, thus being able to classify system states and detect anomalies. Similarly to the previous method, the hyperparameters were chosen with a grid search algorithm based on the highest detection TPR.

ConvAE uses a series of convulsional autoencoders to extract features from the input data, t-SNE to obtain two-dimensional vectors from the extracted features, and the Kmeans algorithm to assign samples with cluster labels. In addition, ConvAE used data mining techniques to select features, structure, and parameters for the autoencoder. Hyperparameter values were equal to the original paper [63] because the selection of hyperparameters for the TEP datasets was done in the original ConvAE paper.

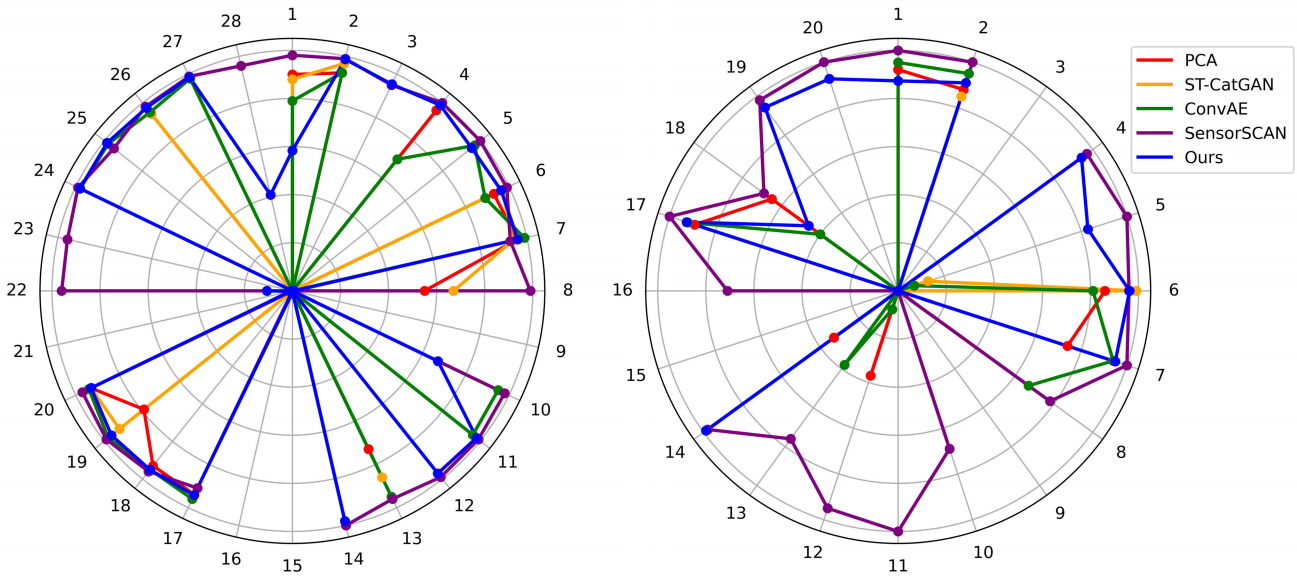


FIGURE 5. Radar charts depicting TPR values for TEP_{Ricker} (left) and TEP_{Rieth} (right), evaluated using four baselines and SensorDBSCAN. Faults are arranged around the circle, with the distance from the center representing TPR values. Charts show that all of described methods fail to detect number of fault types, furthermore, these types are similar for all methods. Proposed method shows performance similar to described baselines in most of fault types.

TABLE 1. Clustering metrics on TEP_{Rieth}. Three of four baselines are outperformed completely with only 20% of training dataset other methods had, our method shows similar performance to the fourth one [2].

Method	ACC	ARI	NMI
PCA+Kmeans	0.274	0.110	0.363
ST-CatGAN	0.175	0.113	0.222
ConvAE	0.179	0.156	0.253
SensorSCAN	**0.785**	0.703	**0.846**
SensorDBSCAN	0.739	**0.712**	0.803

TABLE 2. Clustering metrics on TEP_{Ricker}. Our method outperformed most of the baselines with only 20% of training data, excluding [2] by NMI.

Method	ACC	ARI	NMI
PCA+Kmeans	0.351	0.131	0.448
ST-CatGAN	0.301	0.128	0.360
ConvAE	0.497	0.238	0.586
SensorSCAN	0.736	0.481	**0.850**
SensorDBSCAN	**0.738**	**0.541**	0.815

TABLE 3. Classification metrics on TEP_{Rieth}. First three baselines are outperformed, [2] shows better performance with higher FPR. We were able to achieve good TPR with 0 FPR with only 20% of training dataset compared to other methods, which makes our method the best one within others with 0 FPR.

Method	Detection TPR	Detection FPR	CDR	ADD
PCA+Kmeans	0.3590	**0.0000**	0.7910	114
ST-CatGAN	0.3044	**0.0000**	0.3238	103
ConvAE	0.3631	0.0008	0.3664	165
SensorSCAN	**0.8400**	0.0002	**0.9200**	**5.21**
SensorDBSCAN	0.6877	**0.0000**	0.6913	121

SensorSCAN has two main training steps: self-supervised pre-training with multiple training objectives and SCAN-based deep clustering [4]. Pretraining consists of continuous learning and masked data reconstruction tasks that utilize pseudo-labels produced from original samples with two sets of augmentations. SCAN is a deep clustering technique based on the idea of neighboring points in the embedding space belonging to the same cluster, which is used as an optimization criterion for training, the key difference is demonstrated in Figure 1. The last stage used in the original SensorSCAN paper [2] was supervised fine-tuning.

TABLE 4. Classification metrics on TEP_{Ricker}. Proposed method outperformed three out of four baselines completely. SensorSCAN has shown better TPR with non-zero FPR, than our method with 0 FPR trained with 20% of data.

Method	Detection TPR	Detection FPR	CDR	ADD
PCA+Kmeans	0.3581	**0.0000**	0.9562	113
ST-CatGAN	0.3627	0.0001	0.8882	135
ConvAE	0.6023	0.0016	0.9402	155
SensorSCAN	**0.8700**	0.0002	**0.9600**	**28**
SensorDBSCAN	0.7808	**0.0000**	0.8433	165

E. RESULTS DISCUSSION

Tables 1, 2, 3, 4 show that our method outperformed unsupervised methods in terms of clustering metrics, although it showed slightly worse classification metrics. We used only 20% of the training subset while manually labeling no more than 750 samples (50 samples per epoch for 15 epochs); other methods used 100%. Figure 5 shows a comparison of TPR for each type of faults. We believe that this small number of manually labeled samples makes our method highly useful for real-world applications, as it achieves better performance than unsupervised methods and requires a tiny portion of the dataset to be labeled manually. Furthermore, we believe that an expert can provide enough labels for our algorithm in a few days of manual labeling.

Another important result achieved is the organization of the descriptor space obtained, which is shown in Figures 3

TABLE 5. Key hyperparameters ablation study for TEP_{Ricker}. We consider Detection TPR main performance metric, which makes result observed in experiment with Euclidean distance metric, $\epsilon_t = 1.0$ and *minPts* = 900 the best one. However, cosine distance allows to significantly reduce ADD while preserving 0 Detection FPR and good clustering metrics, though, it gives slightly lower Detection TPR.

Distance metric	ϵ_t	minPts	Detection TPR	Detection FPR	ADD	CDR	ACC	ARI	NMI
Euclidean	1.0	300	0.716	0.014	120	0.911	0.694	0.444	0.775
Euclidean	1.0	600	0.756	0.000	147	0.803	0.713	0.505	0.803
Euclidean	1.0	900	**0.781**	0.000	165	0.843	0.738	**0.541**	0.815
Cosine	0.1	300	0.730	0.000	38	**0.935**	0.744	0.481	**0.816**
Cosine	0.1	600	0.767	0.000	**33**	0.877	**0.757**	0.528	0.815
Cosine	0.1	1200	0.684	0.000	56	0.878	0.467	0.418	0.770

and 4: the visualized decomposition of the descriptor space becomes less noisy and well separated through the training process, while the Calinsky-Harabasz score increases and the outlier percentage reduces.

F. CORE HYPERPARAMETERS DISCUSSION

We conducted a series of experiments to estimate the best hyperparameters for our method, which gave us a number of conclusions. First, tuning ϵ_t cannot improve the results with a fixed distance function, however, various distance functions turned out to work with different values of ϵ_t. Secondly, DBSCAN *minPts* parameter tuning may significantly impact method performance. And finally, every distance function has its own points of good and bad influence on method performance: we have achieved the best detection TPR value with Euclidean distance and 5 times lower ADD with the cosine distance function. Table 5 presents the six best experimental results.

V. CONCLUSION

In this paper, we have proposed the semi-supervised method comprising strengths from both supervised and unsupervised methods - good performance without the need for large amounts of labeled data. Furthermore, we have shown the efficiency of triplet-based contrastive learning, which is widely used in facial image recognition tasks: it is capable of separable descriptor space organization, which makes available the use of simple density-based clustering algorithms. Although our method is not as effective as some other methods, as shown in the tables above, we analyzed and highlighted its advantages:

1) First, our method is capable of achieving decent performance with less than a thousand labeled samples; it combines strengths from both supervised and unsupervised methods.
2) Second, we do not need to perform manual cluster analysis and estimate the number of clusters, which is typical for unsupervised methods. The descriptor space is well organized during training and can be successfully clustered with simple density-based methods that do not require a predefined number of clusters.
3) Finally, we have successfully combined active learning and data augmentation techniques to further improve the data efficiency of our method.

As a result, we were able to achieve decent performance using less than a thousand labeled samples, with a total dataset size of millions of samples. This demonstrated the potential of semi-supervised, active, and contrastive learning.

A. FUTURE WORK

There remains research potential in the field of semisupervised FDD methods and, particularly, our method. We have comprised a list of possible future contributions in this field. First, we plan to conduct a large number of experiments to investigate the method's ability to successfully handle new system states that are not seen in the training data. This ablation study will demonstrate the full potential of the proposed approach. Secondly, as shown in Figure 4, the organization of the descriptor space is partially complete during the first unsupervised training phase, which means that improving the pretraining methodology leads to better performance. Third, we plan improvements in self-supervised learning. In addition, we intend to develop a technique for automatically selecting hyperparameters for clustering algorithms using labeled data. Lastly, our objective is to explore methods for automatically labeling selected samples using a sophisticated pretrained model so that we can simplify real-world application of the suggested approach.

B. LIMITATIONS

Even though the proposed method and baseline have shown decent performance, people should always carefully apply AI-powered systems in real world facilities, especially those that have the potential for natural, social or other types of impact on humanity. It is strongly recommended to monitor AI systems throughout their entire period of operation and to implement fail-safe measures around them.

ACKNOWLEDGMENT

(Petr Ivanov and Ilya Makarov are co-first authors.)

REFERENCES

[1] J. Jennings, N. E. E. Alliance, J. Loveland, and I. P. Sound, "Collaborating with architecture firms to influence design of high performance buildings," in *Proc. ACEEE*, Jan. 2010, pp. 113–124.

[2] M. Golyadkin, V. Pozdnyakov, L. Zhukov, and I. Makarov, "SensorSCAN: Self-supervised learning and deep clustering for fault diagnosis in chemical processes," *Artif. Intell.*, vol. 324, Nov. 2023, Art. no. 104012.

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, Jun. 2017, pp. 5998–6008.

[4] W. V. Gansbeke, S. Vandenhende, S. Georgoulis, M. Proesmans, and L. V. Gool, "SCAN: Learning to classify images without labels," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, May 2020, pp. 268–285.

[5] A. Ram, S. Jalal, A. S. Jalal, and M. Kumar, "A density based algorithm for discovering density varied clusters in large spatial databases," *Int. J. Comput. Appl.*, vol. 3, no. 6, pp. 1–4, Jun. 2010.

[6] K. Tidriri, N. Chatti, S. Verron, and T. Tiplica, "Model-based fault detection and diagnosis of complex chemical processes: A case study of the Tennessee eastman process," *Proc. Inst. Mech. Eng., I, J. Syst. Control Eng.*, vol. 232, no. 6, pp. 742–760, Jul. 2018.

[7] V. S. B. Kurukuru, M. A. Khan, and R. Singh, "Health monitoring framework for electric vehicle drive train in digital twin," in *Proc. 25th Eur. Conf. Power Electron. Appl. (EPE ECCE Eur.)*, Sep. 2023, pp. 1–10.

[8] A. Kovalenko, V. Pozdnyakov, and I. Makarov, "Graph neural networks with trainable adjacency matrices for fault diagnosis on multivariate sensor data," *IEEE Access*, vol. 12, pp. 152860–152872, 2024.

[9] F. Hartung, B. J. Franks, T. Michels, D. Wagner, P. Liznerski, S. Reithermann, S. Fellenz, F. Jirasek, M. Rudolph, D. Neider, H. Leitte, C. Song, B. Kloepper, S. Mandt, M. Bortz, J. Burger, H. Hasse, and M. Kloft, "Deep anomaly detection on Tennessee eastman process data," *Chem. Ingenieur Technik*, vol. 95, no. 7, pp. 1077–1082, Jul. 2023.

[10] M. Ma, L. Han, and C. Zhou, "BTAD: A binary transformer deep neural network model for anomaly detection in multivariate time series data," *Adv. Eng. Informat.*, vol. 56, Apr. 2023, Art. no. 101949.

[11] S. Tuli, G. Casale, and N. R. Jennings, "TranAD: Deep transformer networks for anomaly detection in multivariate time series data," 2022, *arXiv:2201.07284*.

[12] Q. Xiong, Y. Ren, and F. Yang, "Self-attention soft sensor model based on long-short time span," in *Proc. 5th Int. Conf. Ind. Artif. Intell. (IAI)*, Aug. 2023, pp. 1–6.

[13] Y. Lu and J. Li, "CASTrans unsupervised timing anomaly detection networks," in *Proc. 4th Int. Conf. Electr. Eng. Control Technol. (CEECT)*, Dec. 2022, pp. 374–378.

[14] I. Zorin and I. Makarov, "Anomaly detection with self-supervised audio embeddings," in *Proc. DCASE Challenge*, Jul. 2022. [Online]. Available: https://dcase.community/challenge2022/task-unsupervised-anomalous-sound-detection-for-machine-condition-monitoring-results

[15] H. Xu, T. Ren, Z. Mo, and X. Yang, "A fault diagnosis model for Tennessee Eastman processes based on feature selection and probabilistic neural network," *Appl. Sci.*, vol. 12, no. 17, p. 8868, Sep. 2022.

[16] M. Šprogar and D. Verber, "Accuracy is not enough: Optimizing for a fault detection delay," *Mathematics*, vol. 11, no. 15, p. 3369, Aug. 2023.

[17] S. Qin, Y. Luo, and G. Tao, "Memory-augmented U-transformer for multivariate time series anomaly detection," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Jun. 2023, pp. 1–5.

[18] B. Wu, Z. Yao, Y. Tu, and Y. Chen, "A dilated transformer network for time series anomaly detection," in *Proc. IEEE 34th Int. Conf. Tools with Artif. Intell. (ICTAI)*, Oct. 2022, pp. 48–52.

[19] Y. Zhan and R. Yang, "TransVAT: Transformer encoder with variational attention for few-shot fault diagnosis," in *Proc. CAA Symp. Fault Detection, Supervision Saf. Tech. Processes (SAFEPROCESS)*, Sep. 2023, pp. 1–6.

[20] R. Chen, X. Huang, L. Yang, X. Xu, X. Zhang, and Y. Zhang, "Intelligent fault diagnosis method of planetary gearboxes based on convolution neural network and discrete wavelet transform," *Comput. Ind.*, vol. 106, pp. 48–59, Apr. 2019.

[21] C. Wu, P. Jiang, C. Ding, F. Feng, and T. Chen, "Intelligent fault diagnosis of rotating machinery based on one-dimensional convolutional neural network," *Comput. Ind.*, vol. 108, pp. 53–61, Jun. 2019.

[22] J. Jiao, M. Zhao, J. Lin, and K. Liang, "A comprehensive review on convolutional neural network in machine fault diagnosis," *Neurocomputing*, vol. 417, pp. 36–63, Dec. 2020.

[23] S. Chen, J. Yu, and S. Wang, "One-dimensional convolutional neural network-based active feature extraction for fault detection and diagnosis of industrial processes and its understanding via visualization," *ISA Trans.*, vol. 122, pp. 424–443, Mar. 2022.

[24] D. Wei, K. Wang, S. Heyns, and M. J. Zuo, "Convolutional neural networks for fault diagnosis using rotating speed normalized vibration," in *Proc. 6th Int. Conf. Condition Monit. Machinery Non-Stationary Operation*, Santander, Spain. Cham, Switzerland: Springer, 2019, pp. 67–76.

[25] I. Makarov, M. Bakhanova, S. Nikolenko, and O. Gerasimova, "Self-supervised recurrent depth estimation with attention mechanisms," *PeerJ Comput. Sci.*, vol. 8, p. e865, Jan. 2022.

[26] A. Karpov and I. Makarov, "Exploring efficiency of vision transformers for self-supervised monocular depth estimation," in *Proc. IEEE Int. Symp. Mixed Augmented Reality (ISMAR)*, Oct. 2022, pp. 711–719.

[27] A. Luginov and I. Makarov, "SwiftDepth: An efficient hybrid CNN-transformer model for self-supervised monocular depth estimation on mobile devices," in *Proc. IEEE Int. Symp. Mixed Augmented Reality Adjunct (ISMAR-Adjunct)*, Oct. 2023, pp. 642–647.

[28] I. Indyk and I. Makarov, "MonoVAN: Visual attention for self-supervised monocular depth estimation," in *Proc. IEEE Int. Symp. Mixed Augmented Reality (ISMAR)*, Oct. 2023, pp. 1211–1220.

[29] A. Amatov, D. Lamanov, M. Titov, I. Vovk, I. Makarov, and M. Kudinov, "A semi-supervised deep learning approach to dataset collection for query-By-Humming task," in *Proc. ISMIR*, Jan. 2023, pp. 649–656.

[30] F. Cordoni, G. Bacchiega, G. Bondani, R. Radu, and R. Muradore, "A deep learning unsupervised approach for fault diagnosis of household appliances," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 10749–10754, 2020.

[31] Y. Yang, H. Shi, Y. Tao, Y. Ma, B. Song, and S. Tan, "A semi-supervised feature contrast convolutional neural network for processes fault diagnosis," *J. Taiwan Inst. Chem. Eng.*, vol. 151, Oct. 2023, Art. no. 105098.

[32] L. M. Elshenawy, M. A. Halawa, T. A. Mahmoud, H. A. Awad, and M. I. Abdo, "Unsupervised machine learning techniques for fault detection and diagnosis in nuclear power plants," *Prog. Nucl. Energy*, vol. 142, Dec. 2021, Art. no. 103990.

[33] S. K. Lee, H. Kim, M. Chae, H. J. Oh, H. Yoon, and B. D. Youn, "Self-supervised feature learning for motor fault diagnosis under various torque conditions," *Knowl.-Based Syst.*, vol. 288, Mar. 2024, Art. no. 111465.

[34] J. Xu, H. Wu, J. Wang, and M. Long, "Anomaly transformer: Time series anomaly detection with association discrepancy," 2021, *arXiv:2110.02642*.

[35] D. Cacciarelli, M. Kulahci, and J. Tyssedal, "Online active learning for soft sensor development using semi-supervised autoencoders," 2022, *arXiv:2212.13067*.

[36] M. Shajari, H. Geng, K. Hu, and A. Leon-Garcia, "Tensor-based online network anomaly detection and diagnosis," *IEEE Access*, vol. 10, pp. 85792–85817, 2022.

[37] N. Belacel, R. Richard, and Z. M. Xu, "An LSTM encoder–decoder approach for unsupervised online anomaly detection in machine learning packages for streaming data," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Dec. 2022, pp. 3348–3357.

[38] E. Mosqueira-Rey, E. Hernández-Pereira, D. Alonso-Ríos, J. Bobes-Bascarán, and Á. Fernández-Leal, "Human-in-the-loop machine learning: A state of the art," *Artif. Intell. Rev.*, vol. 56, no. 4, pp. 3005–3054, Apr. 2023.

[39] M. Zakariah and A. S. Almazyad, "Anomaly detection for IoT systems using active learning," *Appl. Sci.*, vol. 13, no. 21, p. 12029, Nov. 2023.

[40] A. V. Sadr, B. A. Bassett, and E. Sekyi, "Personalized anomaly detection using deep active learning," *RAS Techn. Instrum.*, vol. 2, no. 1, pp. 586–598, Jan. 2023.

[41] Z. Han and J. Wang, "An adaptive active learning method for fault diagnosis," in *Proc. 35th Chin. Control Conf. (CCC)*, Jul. 2016, pp. 6421–6425.

[42] J. A. Trujillo, I. De-La-Bandera, J. Burgueño, D. Palacios, E. Baena, and R. Barco, "Active learning methodology for expert-assisted anomaly detection in mobile communications," *Sensors*, vol. 23, no. 1, p. 126, Dec. 2022.

[43] J. Bromley, I. Guyon, Y. LeCun, E. Säckinger, and R. Shah, "Signature verification using a 'Siamese time delay neural network," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 6, J. Cowan, G. Tesauro, and J. Alspector, Eds., San Mateo, CA, USA: Morgan Kaufmann, 1993.

[44] H. Wu, Z. Xu, J. Zhang, W. Yan, and X. Ma, "Face recognition based on convolution Siamese networks," in *Proc. 10th Int. Congr. Image Signal Process., Biomed. Eng. Informat. (CISP-BMEI)*, Oct. 2017, pp. 1–5.

[45] I. Melekhov, J. Kannala, and E. Rahtu, "Siamese network features for image matching," in *Proc. 23rd Int. Conf. Pattern Recognit. (ICPR)*, Dec. 2016, pp. 378–383.

[46] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, Feb. 2024, Art. no. 127063.

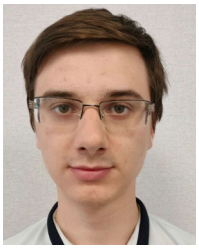
[47] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. 32nd Int. Conf. Int. Conf. Mach. Learn.*, vol. 37, Jan. 2015, pp. 448–456.

[48] K. Sohn, "Improved deep metric learning with multi-class N-pair loss objective," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 29, Dec. 2016, pp. 1857–1865.

[49] M. Ankerst, M. M. Breunig, H.-P. Kriegel, and J. Sander, "OPTICS: Ordering points to identify the clustering structure," *ACM SIGMOD Rec.*, vol. 28, no. 2, pp. 49–60, Jun. 1999.
[50] E. Fix, *Discriminatory Analysis: Nonparametric Discrimination, Consistency Properties*, vol. 1. Wright-Patterson Air Force Base (AFB), OH, USA: USAF School of Aviation Medicine, 1985.
[51] J. J. Downs and E. F. Vogel, "A plant-wide industrial process control problem," *Comput. Chem. Eng.*, vol. 17, no. 3, pp. 245–255, Mar. 1993.
[52] P. R. Lyman and C. Georgakis, "Plant-wide control of the Tennessee Eastman problem," *Comput. Chem. Eng.*, vol. 19, no. 3, pp. 321–331, Mar. 1995.
[53] L. H. Chiang, E. L. Russell, and R. D. Braatz, "Fault diagnosis in chemical processes using Fisher discriminant analysis, discriminant partial least squares, and principal component analysis," *Chemometric Intell. Lab. Syst.*, vol. 50, no. 2, pp. 243–252, Mar. 2000.
[54] N. L. Ricker, "Decentralized control of the Tennessee Eastman challenge process," *J. Process Control*, vol. 6, no. 4, pp. 205–221, Aug. 1996.
[55] N. L. Ricker and J. H. Lee, "Nonlinear modeling and state estimation for the Tennessee Eastman challenge process," *Comput. Chem. Eng.*, vol. 19, no. 9, pp. 983–1005, Sep. 1995.
[56] T. Larsson, K. Hestetun, E. Hovland, and S. Skogestad, "Self-optimizing control of a large-scale plant: The Tennessee eastman process," *Ind. Eng. Chem. Res.*, vol. 40, no. 22, pp. 4889–4901, Oct. 2001.
[57] H. Fan, F. Zhang, R. Wang, X. Huang, and Z. Li, "Semi-supervised time series classification by temporal relation prediction," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Jun. 2021, pp. 3545–3549.
[58] C. Reinartz, M. Kulahci, and O. Ravn, "An extended Tennessee Eastman simulation dataset for fault-detection and decision support systems," *Comput. Chem. Eng.*, vol. 149, Jun. 2021, Art. no. 107281.
[59] L. Hubert and P. Arabie, "Comparing partitions," *J. Classification*, vol. 2, no. 1, pp. 193–218, Dec. 1985.
[60] C. E. Shannon, "A mathematical theory of communication," *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, Jul. 1948.
[61] S. Raschka, J. Patterson, and C. Nolet, "Machine learning in Python: Main developments and technology trends in data science, machine learning, and artificial intelligence," 2020, *arXiv:2002.04803*.
[62] H. Tao, P. Wang, Y. Chen, V. Stojanovic, and H. Yang, "An unsupervised fault diagnosis method for rolling bearing using STFT and generative neural networks," *J. Franklin Inst.*, vol. 357, no. 11, pp. 7286–7307, Jul. 2020.
[63] S. Zheng and J. Zhao, "A new unsupervised data mining method based on the stacked autoencoder for chemical process fault diagnosis," *Comput. Chem. Eng.*, vol. 135, Apr. 2020, Art. no. 106755.
[64] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," 2015, *arXiv:1511.06434*.

PETR IVANOV received the bachelor's degree in software engineering from Ural Federal University, Yekaterinburg, Russia. Currently, he is pursuing the master's degree in artificial intelligence with ITMO University, Saint Petersburg, Russia. He is a Researcher with the Research and Development Department, Software Development Company, Russia.

MARIA SHTARK received the bachelor's degree in software engineering from Ural Federal University, Yekaterinburg, Russia. She is currently pursuing the master's degree in artificial intelligence with ITMO University, Saint Petersburg, Russia. She is a ML Developer in a computer-vision-related company in Russia.



ALEXANDER KOZHEVNIKOV received the bachelor's degree in software engineering from Ural Federal University, Yekaterinburg, Russia. He is currently pursuing the master's degree in artificial intelligence with ITMO University, Saint Petersburg, Russia. He is a Researcher with the Research and Development Department, Software Development Company, Russia.

MAKSIM GOLYADKIN received the master's degree in applied mathematics and informatics from HSE University, where he is currently pursuing the Ph.D. degree in computer science. He is a Researcher in the field of neural networks applications with the Industrial AI, Artificial Intelligence Research Institute, Moscow, Russia.

DMITRY BOTOV received the Ph.D. degree in artificial intelligence, in 2019. He has over 12 years of experience in data science and data engineering. He founded AI Talent Hub community. Currently, he is an Assistant Professor and the Course Leader of CS&AI Program (Neapolis University Pafos). His research interests focus on generative artificial intelligence, natural language processing techniques, large language models, and the application of machine learning methods in life science.

ILYA MAKAROV (Member, IEEE) received the Specialist degree in mathematics from Lomonosov Moscow State University, Moscow, Russia, in 2011, and the Ph.D. degree in computer science from the University of Ljubljana, Ljubljana, Slovenia, in 2021.

Since 2011, he has been a Lecturer with the School of Data Analysis and Artificial Intelligence, HSE University, Moscow, Russia, where from 2012 to 2016, he was the School Deputy Head. He was also the Program Director of the BigData Academy MADE, VK, and a Researcher with Samsung-PDMI Joint AI Center, St. Petersburg Department of V.A, Steklov Mathematical Institute, Russian Academy of Sciences, Saint Petersburg, Russia. He is currently an Associate Professor and a Senior Research Fellow. He is a Senior Research Fellow with the Artificial Intelligence Research Institute (AIRI), Moscow, where he leads research on industrial AI.